

4.4 自定义基于 PyTorch 的 CPU 算子

4.4.1 实验目的

掌握如何在 PyTorch 中新增自定义的 `mysigmoid` 算子。具体包括：

- 1) 熟悉 PyTorch 整体设计机理；
- 2) 通过在 PyTorch 框架中添加自定义的 `mysigmoid` 算子，加深对 PyTorch 算子实现机制的理解，以在 PyTorch 中添加自定义 CPU 算子为例，为后续在 PyTorch 中添加自定义的 DLP 算子奠定基础。

实验工作量：约 50 行代码，约需 2.5 个小时。

4.4.2 背景介绍

4.4.2.1 sigmoid 介绍

激活函数是深度神经网络中对神经元线性加权结果的非线性处理，赋予神经网络非线性映射能力。常见的激活函数有 Sigmoid、Tanh、ReLU、Leaky ReLU 等。其中，Sigmoid 函数具有以下优点：

- 1) 输出值在 0 到 1 之间，便于观察模型输出是否符合预期；
- 2) Sigmoid 函数的导数在 0 到 0.25 之间，可以控制反向传播过程中权重的更新幅度；
- 3) Sigmoid 函数的输出接近 0 或 1 时，输出的导数接近 0，可以避免梯度消失的问题。

本实验自定义的 `mysigmoid` 算子参照 Sigmoid 激活函数实现，其具体计算公式如下：

$$\text{mysigmoid}(x) = \frac{1}{1 + e^{-x}} \quad (4.7)$$

4.4.2.2 在 PyTorch 中添加算子的流程

深度学习算法由多个计算操作组成，我们将这些计算操作称为算子（Operator，简称 OP）。在 PyTorch 中添加新算子主要分为三种方式：基于原生 PyTorch 的方式、基于 C++ 扩展的方式以及基于 OP Register 的方式。

其中，基于原生 PyTorch 的方式是 PyTorch 原生算子的组织方式。这种方式可以比较便捷地新增或者修改算子，但是需要在 PyTorch 源码中修改，与 PyTorch 耦合度高，每次新增或修改都要重新编译 PyTorch。C++ 扩展以及 OP Register 的方式与 PyTorch 解耦，新增算子不需要修改 PyTorch 源码。本实验对 C++ 扩展方式进行介绍，该方式通常包括以下三个步骤：

- 1) C++ 实现。需要用 C++ 实现算子的具体功能。算子的实现称为核函数（Kernel Function）。针对不同的输入、输出类型或硬件架构（如 CPU、GPU、DLP 等），可以有多个 Kernel 实现。
- 2) 算子编译。将 C++ 算子实现编译为库文件的方式主要有三种：通过 CMake 编译、通过 Torch 的 JIT 编译和通过 Setuptools 编译。本实验采用 Setuptools 编译方式。
- 3) 算子测试。通过编写测试程序验证 C++ 实现的算子是否添加成功。

4.4.3 实验环境

硬件平台：MLU 云平台环境（只使用 CPU）。

软件环境：编程框架 PyTorch、Python 3 及相关的扩展库 Pillow、Numpy、Scipy。

4.4.4 实验内容

基于第4.2节实时风格迁移推理实验，我们将其中的 Sigmoid 算子替换为自定义的 `mysigmoid` 算子。该实验不仅涉及使用 C++ 扩展的方式实现 `mysigmoid` 算子的添加，还需要在推理文件中调用 `mysigmoid` 算子，并进行测试。本节实验内容如图 4.4 所示，主要流程包括：

- 1) C++ 实现：编写 `mysigmoid` 算子对应的 C++ 扩展文件；

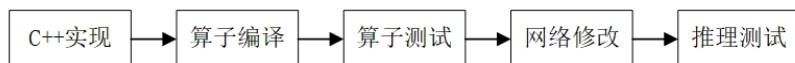


图 4.4 自定义 mysigmoid 算子添加实验流程图

- 2) 算子编译：使用 Setuptools 进行算子编译；
- 3) 算子测试：通过测试程序验证 mysigmoid 算子是否添加成功；
- 4) 网络修改：将网络结构中的 Sigmoid 算子更改为自定义的 mysigmoid 算子；
- 5) 推理测试：运行添加 mysigmoid 算子的实时风格迁移代码，完成实验推理。

4.4.5 实验步骤

如前所述，完整的实验流程主要包括：算子的 C++ 实现、算子编译、算子测试、网络修改及推理测试 5 部分。

4.4.5.1 算子的 C++ 实现

自定义 mysigmoid 算子的 C++ 实现代码如代码示例 4.14 所示，包含头文件引用、mysigmoid 算子的具体实现和算子与 PyTorch 模块的绑定三个部分，具体包括：

- 1) 头文件引用：torch/extension.h 头文件是编写 PyTorch C++ 扩展时必须包含的文件。它基本上囊括了实现所需的所有依赖，包括 ATen 库（PyTorch 底层张量运算库），pybind11（实现 C++ 代码到 Python 的绑定）及二者的交互。
- 2) mysigmoid 算子的具体实现：先将传入的 Tensor 数据转换为 Vector，再根据 mysigmoid 的计算公式计算结果，最后将输出数据重新转换为 Tensor 类型，并将形状重塑为 [1, 3, 512, 512]；
- 3) 算子与 PyTorch 模块的绑定：需添加 pybind11 的绑定说明。这是由于 PyTorch C++ 扩展是通过 pybind11 绑定到 Python 的。在 m.def 中传入参数，分别是绑定到 Python 的函数名称、需绑定的 C++ 函数引用及一个简短的函数说明字符串，用来添加到 Python 函数中的 __doc__ 成员中。

代码示例 4.14 自定义 mysigmoid 算子的 C++ 实现

```

1 // file: mysigmoid.cpp
2 // Pytorch 扩展头文件的引用
3 #include <torch/extension.h>
4 using namespace std;
5
6 // mysigmoid_cpu 函数的具体实现
7 torch::Tensor mysigmoid_cpu(const torch::Tensor & dets) {
8     //TODO: 将输入的 tensor 转化为浮点类型的 vector
9
10     int input_size = input_data.size();
11     //TODO: 创建一个浮点类型的 output_data，output_data 为大小与输入相同的 vector
12
13     //TODO: 对于输入向量的每个元素计算 mysigmoid
14
15     //TODO: Create tensor options with dtype float32
16     auto opts = torch::TensorOptions().dtype(torch::kFloat32);
17     //TODO: Create a tensor from the output vector
18     auto foo = torch::from_blob(output_data.data(), {int64_t(output_data.size())}, opts).clone();
  
```

```

19  //TODO: 将得到的tensor转换成所需的大小
20  -----
21  return output;
22  }
23  //TODO: 算子绑定为Pytorch的模块
24  PYBIND11_MODULE(TORCH_EXTENSION_NAME, m) { // 绑定部分
25  -----
26  }

```

4.4.5.2 算子编译

本实验利用 Python 中提供的 Setuptools 模块完成编译流程，将包含算子的 C++ 文件，编译为一个动态链接库（在 Linux 平台是一个 .so 后缀文件），可以让 Python 调用其中实现的函数功能。调用 Setuptools 的 setup.py 文件的代码示例如 4.15 所示。在实验中，需要补全 setuptools.setup 函数中编译链接的库名称，通过 cpp_extension 包含待编译的文件及编译函数，并配置编译命令。

补全好以上代码后，执行 `python setup.py build_ext -- inplace` 命令进行编译，从而生成 .so 动态链接文件。

代码示例 4.15 算子编译代码示例

```

1  ## file: setup.py
2  from setuptools import setup
3  from torch.utils import cpp_extension
4
5  setup(
6      #TODO: 给出编译后的链接库名称
7      -----
8      ext_modules=[
9          cpp_extension.CppExtension(
10             #TODO: 以正确的格式给出编译文件即编译函数
11             -----
12             )
13     ],
14     # 执行编译命令设置
15     cmdclass={
16         'build_ext': cpp_extension.BuildExtension
17     }
18 )
19
20 print("generate .so PASS!\n")

```

4.4.5.3 算子测试

为了验证 Python 是否可以正确导入动态链接库中的 `mysigmoid` 算子，通过测试程序 `test_mysigmoid.py` 对算子进行测试。具体而言，首先导入一些必要的库和模块，并定义在 CPU 上执行的 `mysigmoid` 函数及其测试函数。在测试函数中生成一个形状为 (3, 512, 512) 的随机张量，调用 `mysigmoid_cpu` 函数对其进行处理得到输出结果。最后打印输入张量和输出结果的形状。`test_mysigmoid.py` 测试程序的代码示例如 4.16 所示，需要按照说明要求进行补全并测试。

代码示例 4.16 算子测试代码示例

```

1  ## file: test_mysigmoid.py
2  import numpy as np
3  import torch
4  import torchvision
5  import numpy as np
6  #TODO: 导入自定义连接库
7
8  -----
9  def mysigmoid_cpu(rand):
10     rand = rand.contiguous()
11     #TODO: 调用mysigmoid函数对rand进行处理得到输出结果output
12
13     -----
14     return output.contiguous()
15
16 def test_mysigmoid():
17     torch.manual_seed(12345)
18     rand = (torch.randn(3, 512, 512, dtype=torch.float32).abs()+1)
19     #TODO: 调用mysigmoid_cpu函数对rand进行处理得到输出结果output_cpu
20
21     -----
22     print("-----mysigmoid test completed-----")
23     print("input: ", rand)
24     print("input_size:", rand.size())
25     print("output: ", output_cpu)
26     print("output_size:", output_cpu.size())
27
28     print("TEST mysigmoid PASS!\n")
29
30 test_mysigmoid()

```

4.4.5.4 网络修改

本实验基于第4.2节实时风格迁移推理实验，将其中的 Sigmoid 算子替换为自定义的 mysigmoid 算子。因此，在上述步骤完成之后，需要对 evaluate_cpu.py 文件进行相应的修改，如代码示例4.17所示。为方便读者理解，仅展示本实验需要完成的内容。具体而言，需要完成两步：导入自定义动态链接库、在前向传播（Forward）中添加自定义 mysigmoid 算子。

代码示例 4.17 实时风格迁移网络修改示例

```

1  ## file: evaluate_cpu.py
2  .....
3  #TODO: 导入自定义动态链接库
4
5  -----
6  .....
7  class TransNet(nn.Module):
8  .....
9      def forward(self, x):
10         x = self.layer(x)
11         #TODO: 调用自定义mysigmoid算子对x进行处理得到输出结果out
12
13         -----
14         return out
15
16 .....

```

4.4.5.5 模型推理

完成网络修改后，执行 `python evaluate_cpu.py` 运行修改好的代码，查看 `/out/cpu` 目录下的推理结果。

4.4.5.6 实验运行

本实验需要根据第4.4.5.1节 ~ 第4.4.5.5节的描述补全 `mysigmoid.cpp`、`setup.py` 文件，运行 `setup.py` 进行算子编译。在生成动态链接库之后，补全 `test_mysigmoid.py` 进行算子测试，验证 `mysigmoid` 算子是否被正确添加。然后，修改 `evluate_cpu.py` 中的网络结构，并进行模型推理测试，具体步骤如下：

1) 环境申请

申请实验环境并登录云平台。具体而言，本实验的代码存放在云平台中，登录后即可访问。相关代码放于 `/opt/code_chap_4/exp_4_4_mysigmoid/` 目录下。

```
# 登录云平台
ssh root@xxx.xxx.xxx -p xxxxx
# 进入 /opt/code_chap_4/exp_4_4_mysigmoid/ 目录
cd /opt/code_chap_4/exp_4_4_mysigmoid/
```

2) 代码实现

补全 `mysigmoid.cpp`、`setup.py`、`test_mysigmoid.py`、`evluate_cpu.py` 文件。

```
# 进入 op_mysigmoid/ 实验目录
cd stu_upload/op_mysigmoid/
# 补全mysigmoid.cpp文件
vim mysigmoid.cpp
# 补全setup.py文件
vim setup.py

# 进入 stu_upload/ 实验目录
cd stu_upload/
# 补全test_mysigmoid.py文件
vim test_mysigmoid.py
# 补全evluate_cpu.py文件
vim evluate_cpu.py
```

3) 运行实验

补全代码之后，需要在 `op_mysigmoid` 目录下进行算子编译，生成动态链接库后，进入 `stu_upload` 实验目录，使用 `Python` 进行算子测试，并进行模型推理。

```
# 进入 op_mysigmoid/ 实验目录
cd stu_upload/op_mysigmoid/
# 进行算子编译生成动态链接库
python setup.py build_ext --inplace
# 进入 stu_upload/ 实验目录
cd stu_upload/
# 进行算子测试
python test_mysigmoid.py
# 进行模型推理
python evaluate_cpu.py
```

4.4.6 实验评估

本实验的评估标准设定如下：

- 60 分标准：完成基于 C++ 扩展方法在 PyTorch 中添加 mysigmoid 算子的工作，能够正确生成动态链接库。
- 80 分标准：在 60 分基础上，补全算子测试程序，能够正确调用 mysigmoid 算子。
- 100 分标准：在 80 分基础上，修改网络结构，能够正确实现添加 mysigmoid 算子进行实时风格迁移推理。

4.4.7 实验思考

- 1) 框架内部核心代码为何采用 C/C++ 语言，而非高级语言（如 Python 等）？
- 2) PyTorch 的算子添加方式有哪些？各有什么优劣势？
- 3) 将 C++ 算子编译为库文件的方式有哪几种？各有什么优劣势？

4.5 Transformer 架构实现与应用

4.5.1 实验目的

掌握 Transformer 模型的核心原理与 PyTorch 实现方法。通过模块化编程、模型搭建与推理评估，深入理解 Transformer 的工作机制，并具备在 PyTorch 中实现复杂深度学习模型的能力。具体包括：

- 1) 理解 Transformer 模型的整体架构，包括编码器-解码器结构及其关键组件；
- 2) 掌握位置编码、多头注意力、前馈神经网络等核心模块的原理与实现方法；
- 3) 熟悉在 PyTorch 中完成 Transformer 模型搭建与推理的流程；
- 4) 能够在 DLP 上完成翻译任务的性能验证，并具备基础的模型分析能力。

实验工作量：约 90 行代码，约需 4 个小时。

4.5.2 背景介绍

4.5.2.1 机器翻译

机器翻译（Machine Translation, MT）致力于通过计算机系统将源语言自动转换为目标语言，是跨语言交流的重要技术。早期神经机器翻译主要采用序列到序列（Seq2Seq）模型，通过编码器-解码器结构实现句子级的语义建模，如 4.5 所示。随着机器翻译在在线翻译、跨语言搜索和国际交流等应用中需求的广泛增长，传统 Seq2Seq 方法在处理长文本和大规模数据时的局限逐渐显现。为此，基于自注意力机制的 Transformer^[28] 架构应运而生，凭借更高效的并行计算能力和更强的建模能力，成为当前机器翻译系统的主流模型。

4.5.2.2 Transformer 网络结构

Transformer 网络是目前自然语言处理常用的模型之一，其整体结构已经在第 3.4.2.3 小节进行了介绍。本部分详细介绍其词嵌入模块、层归一化模块、位置编码模块、前馈神经

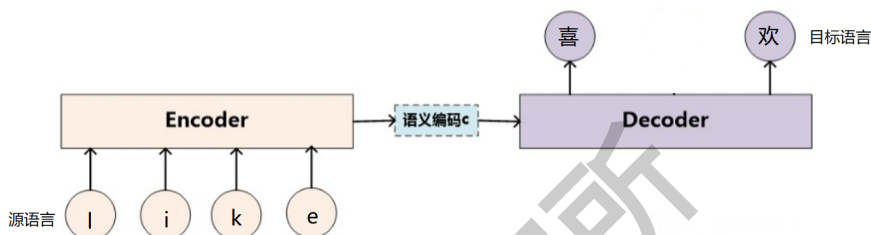


图 4.5 Seq2Seq 模型

网络模块的结构，并对标签平滑技术进行介绍。

1) 词嵌入模块

词嵌入模块用于将输入数据映射为高维稠密向量表示，以方便下一步的计算和处理。该模块支持首行零填充以及按 $\sqrt{d}$ 进行缩放（ d 表示词嵌入维度），有助于提高训练的数值稳定性，是 Transformer 等模型中输入编码的重要组成部分。

2) 层归一化模块

层归一化模块对每个输入样本的每个特征进行归一化，使其均值为 0、方差为 1，并通过引入可学习的缩放因子和偏移因子，对归一化后的输出进行缩放和偏移，从而提高训练的稳定性及收敛速度。其计算公式为：

$$\text{LayerNorm}(x) = \gamma \cdot \frac{x - \mu}{\sigma + \varepsilon} + \beta \quad (4.8)$$

其中， x 表示输入， μ 为输入的均值， σ 为标准差， ε 为防止除零错误的小常数， γ 为缩放因子， β 为偏移因子。

3) 位置编码模块

位置编码模块根据序列的位置生成一个固定的正弦-余弦函数编码，从而为注意力机制引入位置信息，帮助模型理解序列顺序。其核心计算如公式4.9和公式4.10所示。

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right) \quad (4.9)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d}}}\right) \quad (4.10)$$

其中， pos 表示位置， i 是维度索引， d 为编码的总维度。对于偶数维度使用正弦函数编码，奇数维度使用余弦函数编码。

4) 前馈神经网络模块

前馈神经网络模块位于多头注意力机制模块之后，能够对每个位置独立地进行非线性变换。该模块包含两个线性变换，中间通过 ReLU 激活函数。输入先通过第一层线性变换将维度扩展到较高的隐藏维度，再通过第二层线性变换将维度还原回输入维度。该模块中还包含残差连接，将输入直接加到输出上，并在输出端应用层归一化方法，以加快训练收敛并提升模型稳定性。

5) 标签平滑

在 Transformer 网络的损失函数计算过程中,使用了标签平滑 (Label Smoothing) 技术^[72]。这是一种常用的正则化方法,用于防止模型过拟合和提升泛化能力。其核心思想是将原始的独热 (One-hot) 标签转换为更平滑的概率分布,防止模型对某一类别过度自信。计算公式如下:

$$\text{LabelSmoothing}(x) = (1 - \epsilon) \cdot x + \frac{\epsilon}{K} \quad (4.11)$$

其中, ϵ 为平滑系数, x 为原始的标签分布, K 为类别总数。标签平滑将目标类别的概率降低到 $1 - \epsilon + \frac{\epsilon}{K}$, 其余类别的概率均匀分配为 $\frac{\epsilon}{K}$ 。在计算损失函数时, 需要先对真实标签 (目标序列) 进行标签平滑处理, 再计算其与模型预测输出的损失函数。

4.5.3 实验环境

硬件平台: MLU 云平台环境。

软件环境: 编程框架 PyTorch、计算加速库 CNL、云端开发工具集 CNToolkit、Python3 及相关的扩展库。

数据集: IWSLT2016 是国际机器翻译领域广泛使用的公开基准数据集之一, 由 IWSLT (International Workshop on Spoken Language Translation) 组织发布^[73]。该数据集包含英文与中文的双向平行语料, 主要来源于 TED 演讲的字幕文本, 覆盖多种主题和真实口语场景。训练集约包含 5 万对句子, 验证集和测试集各包含上千对句子。由于内容贴近实际口语交流, IWSLT2016 不仅考察模型在正式文本上的表现, 更衡量其在口语、跨领域翻译中的泛化能力。该数据集广泛用于 Transformer 等神经机器翻译模型的训练与性能评估, 支持 BLEU 等主流指标计算, 已成为学术研究与工业应用中的重要评测标准。

4.5.4 实验内容

本实验围绕 Transformer 模型展开, 首先实现基本单元模块, 再完成 Transformer 模型搭建与性能评估。具体包括:

- 1) 实现词嵌入、位置编码、多头注意力等基本单元模块。
- 2) 基于基本单元模块搭建编码器-解码器模型, 并构建损失函数。
- 3) 实现 Transformer 模型推理与 BLEU 评估。

4.5.5 实验步骤

4.5.5.1 Transformer 基本单元模块实现

Transformer 基本单元模块主要由词嵌入模块、层归一化模块、位置编码模块、多头注意力机制模块、前馈神经网络模块、标签平滑模块构成。

1) 词嵌入模块

词嵌入模块如代码示例 4.18 所示。

代码示例 4.18 词嵌入模块

```
1 ## file: modules.py
```

```

2  # -*- coding: utf-8 -*-
3  from __future__ import print_function
4  import torch
5  import torch.nn as nn
6  import torch.nn.functional as F
7  from torch.autograd import *
8  import numpy as np
9  from torch.nn.parameter import Parameter
10 from hyperparams import *
11
12 class embedding(nn.Module):
13
14     def __init__(self, vocab_size, num_units, zeros_pad=True, scale=True):
15         '''Embeds a given Variable.
16         Args:
17             vocab_size: An int. Vocabulary size.
18             num_units: An int. Number of embedding hidden units.
19             zero_pad: A boolean. If True, all the values of the first row (id 0)
20                 should be constant zeros.
21             scale: A boolean. If True, the outputs is multiplied by sqrt num_units.
22         '''
23         super(embedding, self).__init__()
24         self.vocab_size = vocab_size
25         self.num_units = num_units
26         self.zeros_pad = zeros_pad
27         self.scale = scale
28         self.lookup_table = Parameter(torch.Tensor(vocab_size, num_units))
29         #TODO: 使用Xavier正态初始化方法对嵌入矩阵进行初始化
30         _____(self.lookup_table.data)
31         if self.zeros_pad:
32             self.lookup_table.data[0, :].fill_(0)
33
34     def forward(self, inputs):
35         if self.zeros_pad:
36             self.padding_idx = 0
37         else:
38             self.padding_idx = -1
39         #TODO: 调用内置函数将输入的词索引映射为对应的词向量表示，用于实现嵌入层功能
40         outputs = _____(
41             _____, _____, _____, None, 2, False, False)
42
43         if self.scale:
44             outputs = outputs * (self.num_units ** 0.5)
45
46         return outputs

```

2) 层归一化模块

层归一化模块如代码示例4.19所示。在 Transformer 网络中，层归一化用于对每个样本的特征向量（输入的最后一个维度）进行归一化操作，从而提高网络训练的稳定性和收敛速度。在初始化中，设置缩放因子 γ 初值为 1，偏移因子 β 初值为 0。

代码示例 4.19 层归一化模块

```

1  ## file: modules.py
2  class layer_normalization(nn.Module):
3
4     def __init__(self, features, epsilon=1e-8):
5         '''Applies layer normalization.
6
7         Args:
8             epsilon: A floating number. A very small number for preventing ZeroDivision Error.

```

```

9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
'''
super(layer_normalization, self).__init__()
self.epsilon = epsilon
#TODO: 定义可训练的缩放参数 gamma, 初始值全为1, 形状与 features 维度一致
self.gamma = _____
#TODO: 定义可训练的偏移参数 beta, 初始值全为0, 形状与 features 维度一致
self.beta = _____

def forward(self, x):
#TODO: 对输入计算最后一个维度的均值
mean = _____(_____, keepdim=True)
#TODO: 对输入计算最后一个维度的标准差
std = _____(_____, keepdim=True)
#TODO: 对输入进行层归一化
return _____

```

3) 位置编码模块

根据公式4.9和公式4.10实现位置编码, 如代码示例4.20所示。前向传播过程中, 输入为形状 (N, T) 的序列张量, 位置编码模块将其映射为形状 (N, T, d) 的位置编码矩阵。为增强特征表达能力, 该实现支持零填充 (将位置 $pos = 0$ 的编码设为全零) 和缩放 (乘以 $\sqrt{d}$) 以增强特征表达。其中, 编码的总维度 d 在代码中由参数 `num_unit` 指定。

代码示例 4.20 位置编码模块

```

1  ## file: modules.py
2  class positional_encoding(nn.Module):
3
4      def __init__(self, num_units, zeros_pad=True, scale=True):
5          '''Sinusoidal Positional Encoding.
6
7          Args:
8              num_units: Output dimensionality
9              zero_pad: Boolean. If True, all the values of the first row (id = 0) should be constant zero
10             scale: Boolean. If True, the output will be multiplied by sqrt num_units (check details from
11             paper)
12             '''
13             super(positional_encoding, self).__init__()
14             self.num_units = num_units
15             self.zeros_pad = zeros_pad
16             self.scale = scale
17
18             def forward(self, inputs, y):
19                 # inputs: A 2d Tensor with shape of (N, T).
20                 N, T = inputs.size()[0: 2]
21
22                 # First part of the PE function: sin and cos argument
23                 # position_ind = Variable(torch.unsqueeze(torch.arange(0, T), 0).repeat(N, 1).cuda().long())
24                 position_ind = torch.unsqueeze(torch.arange(0, T), 0).repeat(N, 1)
25                 if (inputs.is_cuda):
26                     position_ind = Variable(position_ind.cuda().long())
27                 if (inputs.device.type == 'mlu'):
28                     position_ind = Variable(position_ind.to('mlu').long())
29
30                 #TODO: 根据论文公式, 计算位置编码矩阵, 形状为(T, num_units)
31                 position_enc = torch.Tensor(_____)
32
33                 # Second part, apply the cosine to even columns and sin to odds.
34                 #TODO: 对偶数列 (从0开始) 使用 sin
35                 position_enc[:, 0::2] = _____ # dim 2i

```

```

36 #TODO: 对奇数列使用cos
37 position_enc[:, 1::2] = _____ # dim 2i+1
38
39 # Convert to a Variable
40 # lookup_table = Variable(position_enc).cuda()
41 lookup_table = Variable(position_enc)
42 if (inputs.is_cuda):
43     lookup_table = lookup_table.cuda()
44 if (inputs.device.type == 'mlu'):
45     lookup_table = lookup_table.to('mlu').to(y.dtype)
46
47
48 if self.zeros_pad:
49     lookup_table = torch.cat((Variable(torch.zeros(1, self.num_units)),
50                                lookup_table[1:, :]), 0)
51
52 padding_idx = 0
53 else:
54     padding_idx = -1
55
56 #TODO: 根据位置索引 position_ind, 从位置编码查找表 lookup_table 中取出对应的位置编码向量, 生成
57 #TODO: 最终的编码输出。
58 outputs = _____(
59     _____, padding_idx, None, 2, False, False)
60
61 if self.scale:
62     #TODO: 将输出进行缩放
63     outputs = _____
64
65 return outputs

```

4) 多头注意力机制模块

多头注意力机制如代码示例4.21所示。Q、K、V 矩阵经过缩放点积注意力计算后，多个头的结果需要拼接还原，最后通过残差连接和层归一化后输出。

代码示例 4.21 多头注意力机制模块

```

1 ## file: modules.py
2 class MultiheadAttention(nn.Module):
3
4     def __init__(self, hp_, num_units, num_heads=8, dropout_rate=0, causality=False):
5         '''Applies multihead attention.
6
7         Args:
8             num_units: A scalar. Attention size.
9             dropout_rate: A floating point number.
10             causality: Boolean. If true, units that reference the future are masked.
11             num_heads: An int. Number of heads.
12         '''
13         super(MultiheadAttention, self).__init__()
14         self.hp = hp_
15         self.num_units = num_units
16         self.num_heads = num_heads
17         self.dropout_rate = dropout_rate
18         self.causality = causality
19         #TODO: 构建Q、K、V的线性映射层, 包含全连接和ReLU激活, 用于将输入特征投影到注意力空间
20         self.Q_proj = nn.Sequential(_____, _____)
21         self.K_proj = nn.Sequential(_____, _____)
22         self.V_proj = nn.Sequential(_____, _____)
23         #TODO: 输出 dropout 层
24         self.output_dropout = _____(self.dropout_rate)
25         #TODO: 调用自定义函数实现层归一化, 标准化输出

```

```

26     self.normalization = _____(self.num_units)
27     #self.normalization = nn.LayerNorm(self.num_units, eps=1e-8)
28
29 def forward(self, queries, keys, values):
30     # keys, values: same shape of [N, T_k, C_k]
31     # queries: A 3d Variable with shape of [N, T_q, C_q]
32
33     # Linear projections
34     Q = self.Q_proj(queries) # (N, T_q, C)
35     K = self.K_proj(keys) # (N, T_q, C)
36     V = self.V_proj(values) # (N, T_q, C)
37
38     # Split and concat
39     #TODO: 将Q, K, V按最后一维均分为num_heads份, 并在batch维拼接
40     Q_ = _____ # (h*N, T_q, C/h)
41     K_ = _____ # (h*N, T_q, C/h)
42     V_ = _____ # (h*N, T_q, C/h)
43
44     # Multiplication
45     #TODO: 计算Q与K的转置在每个注意力头内的批量矩阵乘法, 得到注意力得分
46     outputs = _____ # (h*N, T_q, T_k)
47
48     # Scale
49     #TODO: 按键的最后一维度平方根缩放注意力得分
50     outputs = _____
51
52     # Key Masking
53     key_masks = torch.sign(torch.abs(torch.sum(keys, dim=-1))) # (N, T_k)
54     key_masks = key_masks.repeat(self.num_heads, 1) # (h*N, T_k)
55     key_masks = torch.unsqueeze(key_masks, 1).repeat(1, queries.size()[1], 1) # (h*N, T_q, T_k)
56
57     #padding = Variable(torch.ones(*outputs.size()).cuda() * (-2 ** 32 + 1))
58     init_tensor = torch.ones(*outputs.size(), dtype=queries.dtype)
59     if (queries.is_cuda):
60         init_tensor = init_tensor.cuda()
61     if (queries.device.type == 'mlu'):
62         init_tensor = init_tensor.to('mlu')
63     padding = Variable(init_tensor * (-2 ** 32 + 1))
64
65     condition = key_masks.eq(0.)
66     outputs = padding * condition + outputs * (~condition) #pytorch1.2 version incompatibility,
change 1.-b to -b
67
68     # Causality = Future blinding
69     if self.causality:
70         #diag_vals = torch.ones(*outputs[0, :, :].size()).cuda() # (T_q, T_k)
71         diag_vals = torch.ones(*outputs[0, :, :].size(), dtype=queries.dtype) # (T_q, T_k)
72         if (queries.is_cuda):
73             diag_vals = diag_vals.cuda()
74         if (queries.device.type == 'mlu'):
75             diag_vals = diag_vals.to('mlu')
76
77         #TODO: 生成一个下三角矩阵, 主对角线及其以下由 diag_vals 填充, 其余位置为零
78         tril = _____ # (T_q, T_k)
79         # print(tril)
80         masks = Variable(torch.unsqueeze(tril, 0).repeat(outputs.size()[0], 1, 1)) # (h*N, T_q,
T_k)
81
82         #padding = Variable(torch.ones(*masks.size()).cuda() * (-2 ** 32 + 1))
83         mask = torch.ones(*masks.size(), dtype=queries.dtype)
84         if (queries.is_cuda):
85             mask = mask.cuda()

```

```

86         if (queries.device.type == 'mlu'):
87             mask = mask.to('mlu')
88             #TODO: 生成一个下三角矩阵, 主对角线及其以下由 diag_vals 填充, 其余位置为零
89             tril = ----- # (T_q, T_k)
90             padding = Variable(mask * (-2 ** 32 + 1))
91
92             condition = masks.eq(0.)
93             outputs = padding * condition + outputs * (~condition) #pytorch1.2 version incompatibility
, change 1.-b to ~b
94
95             # Activation
96             #TODO: 对最后一个维度做softmax, 计算注意力权重
97             outputs = ----- # (h*N, T_q, T_k)
98
99             # Query Masking
100             query_masks = torch.sign(torch.abs(torch.sum(queries, dim=-1))) # (N, T_q)
101             query_masks = query_masks.repeat(self.num_heads, 1) # (h*N, T_q)
102             query_masks = torch.unsqueeze(query_masks, 2).repeat(1, 1, keys.size()[1]) # (h*N, T_q, T_k)
103             #TODO: 屏蔽无效的 query 位置, 防止其影响注意力结果
104             outputs = -----
105
106             # Dropouts
107             #TODO: 对注意力权重做dropout
108             outputs = ----- # (h*N, T_q, T_k)
109
110             # Weighted sum
111             #TODO: 执行批量矩阵乘法, 将注意力权重与值向量相乘
112             outputs = ----- # (h*N, T_q, C/h)
113
114             # Restore shape
115             #TODO: 将多头的输出按特征维度拼接, 还原为(N, T_q, C)
116             outputs = ----- # (N, T_q, C)
117
118             # Residual connection
119             #TODO: 加入残差连接
120             outputs += -----
121
122             # Normalize
123             #TODO: 对输出进行层归一化
124             outputs = ----- # (N, T_q, C)
125
126             return outputs

```

5) 前馈神经网络模块

前馈神经网络模块相当于对每个位置独立地进行非线性变换, 如代码示例4.22所示。其中线性变换层有两种实现方式: 基于一维卷积和基于全连接。卷积方式需调整输入输出形状, 而全连接方式可直接处理原始张量。

代码示例 4.22 前馈神经网络模块

```

1  ## file: modules.py
2  class feedforward(nn.Module):
3
4      def __init__(self, in_channels, num_units=[2048, 512]):
5          '''Point-wise feed forward net.
6
7          Args:
8              in_channels: a number of channels of inputs
9              num_units: A list of two integers.
10             '''
11             super(feedforward, self).__init__()

```

```

12     self.in_channels = in_channels
13     self.num_units = num_units
14
15     # nn.Linear is faster than nn.Conv1d
16     self.conv = False
17     if self.conv:
18         params = {'in_channels': self.in_channels, 'out_channels': self.num_units[0],
19                   'kernel_size': 1, 'stride': 1, 'bias': True}
20         #TODO: 采用卷积构建第一层线性映射, (构建包含一维卷积和ReLU激活的顺序网络, 卷积参数由字典
        params解包传入)
21         self.conv1 = -----
22         params = {'in_channels': self.num_units[0], 'out_channels': self.num_units[1],
23                   'kernel_size': 1, 'stride': 1, 'bias': True}
24         #TODO: 采用卷积构建第二层线性映射, (定义一个一维卷积层, 卷积参数通过字典params解包传入)
25         self.conv2 = -----
26     else:
27         #TODO: 采用全连接方式实现第一层线性映射
28         self.conv1 = -----
29         #TODO: 采用全连接方式实现第二层线性映射
30         self.conv2 = -----
31         #TODO: 调用自定义函数实现层归一化, 标准化输出
32         self.normalization = -----
33         #self.normalization = nn.LayerNorm(self.in_channels, eps=1e-8)
34
35     def forward(self, inputs):
36         if self.conv:
37             #TODO: 调整输入形状(batch_size, seq_len, channels) -> (batch_size, channels, seq_len)
38             inputs = -----
39             #TODO: 构建第一层线性映射
40             outputs = -----
41             #TODO: 构建第二层线性映射
42             outputs = -----
43
44             #TODO: 残差连接
45             outputs += -----
46
47             # Layer normalization
48             #TODO: 如果是卷积实现, 先进行形状转换再进行层归一化
49             if self.conv:
50                 outputs = -----
51             #TODO: 如果是线性映射, 直接归一化
52             else:
53                 outputs = -----
54
55         return outputs

```

6) 标签平滑模块

标签平滑模块如代码示例4.23所示。根据标签的类别数自动生成平滑后的概率分布, 平滑系数 ϵ 设置为 0.1。

代码示例 4.23 标签平滑模块

```

1  ## file: modules.py
2  class label_smoothing(nn.Module):
3
4      def __init__(self, epsilon=0.1):
5          '''Applies label smoothing. See https://arxiv.org/abs/1512.00567.
6
7          Args:
8              epsilon: Smoothing rate.
9          '''

```

```

10     super(label_smoothing, self).__init__()
11     self.epsilon = epsilon
12
13     def forward(self, inputs):
14         #TODO: 获取类别数量
15         K = _____
16         #TODO: 应用公式进行标签平滑
17         return _____

```

4.5.5.2 Transformer 翻译模型构建

Transformer 翻译模型构建模块如代码示例4.24所示。该模块实现了一个基于 Transformer 结构的 Seq2Seq 模型，可用于机器翻译等任务。模型由编码器和解码器两部分组成，调用了基本单元模块中的嵌入模块、层归一化模块、位置编码模块、多头注意力机制模块、前馈神经网络模块以及标签平滑模块。编码器对输入序列进行编码，提取上下文特征；解码器通过注意力和编码器-解码器注意力生成目标序列。模型支持正弦位置编码和可学习位置嵌入，具备灵活性。最终通过全连接层将隐藏状态映射为目标词汇概率，并采用标签平滑和交叉熵计算损失。

代码示例 4.24 Transformer 翻译模型构建

```

1  # -*- coding: utf-8 -*-
2  from __future__ import print_function
3  import torch
4  import torch.nn as nn
5  import torch.nn.functional as F
6  from torch.autograd import *
7  import numpy as np
8  from modules import *
9  from hyperparams import *
10 from hyperparams import Hyperparams as hp
11
12 class AttModel(nn.Module):
13     def __init__(self, hp_, enc_voc, dec_voc):
14         '''Attention is all you need. https://arxiv.org/abs/1706.03762
15         Args:
16             hp: Hyper Parameters
17             enc_voc: vocabulary size of encoder language
18             dec_voc: vocabulary size of decoder language
19         '''
20         super(AttModel, self).__init__()
21         self.hp = hp_
22
23         self.enc_voc = enc_voc
24         self.dec_voc = dec_voc
25
26         # encoder
27         #TODO: 调用基本单元模块完成编码器的词嵌入，将词索引映射为维度为hidden_units的稠密向量，并进行
缩放
28         self.enc_emb = _____
29         print("Embedding PASS!")
30
31         #TODO: 如果超参数中设置使用正弦位置编码，调用位置编码模块，维度为hidden_units，不使用零填充，
也不进行缩放
32         if self.hp.sinusoid:
33             self.enc_positional_encoding = _____

```



```

34     #TODO: 否则使用可学习的嵌入方式生成位置编码, 词表大小为maxlen, 嵌入维度为hidden_units, 不使用
零填充, 也不进行缩放
35     else:
36         self.enc_positional_encoding =

-----
37     print("PositionEncoding PASS!")
38     #TODO: 定义 dropout 层
39     self.enc_dropout = _____(self.hp.dropout_rate)
40
41     for i in range(self.hp.num_blocks):
42         #TODO: 调用多头注意力机制模块, 隐藏维度为hidden_units, 包含num_heads个注意力头, 使用指定的
dropout 率, 且不使用因果掩码
43         self.__setattr__('enc_self_attention_%d' % i,
-----)
44         #TODO: 调用前馈神经网络模块进行构建, 包含一个隐藏层, 中间层维度通常为输入hidden_units的4
倍, 输出层维度恢复到输入
45         self.__setattr__('enc_feed_forward_%d' % i,
-----)
46
47     print("LayerNormalization PASS!")
48     print("MutiheadAtt PASS!")
49     print("FeedForward PASS!")
50
51     # decoder
52     #TODO: 调用基本单元模块完成解码器的词嵌入, 调用基本单元模块将词索引映射为维度为hidden_units的稠
密向量, 并进行缩放
53     self.dec_emb = _____
54     #TODO: 如果超参数中设置使用正弦位置编码, 调用位置编码模块, 维度为hidden_units, 不使用零填充,
也不进行缩放
55     if self.hp.sinusoid:
56         self.dec_positional_encoding =

-----
57     #TODO: 否则使用可学习的嵌入方式生成位置编码, 词表大小为maxlen, 嵌入维度为hidden_units, 不使用
零填充, 也不进行缩放
58     else:
59         self.dec_positional_encoding =

-----
60     #TODO: 定义 dropout 层
61     self.dec_dropout = _____
62     for i in range(self.hp.num_blocks):
63         #TODO: 调用多头注意力机制模块, 隐藏维度为hidden_units, 包含num_heads个注意力头, 使用指定的
dropout 率, 使用掩码
64         self.__setattr__('dec_self_attention_%d' % i,
-----)
65         #TODO: 调用多头注意力机制模块, 隐藏维度为hidden_units, 包含num_heads个注意力头, 使用指定的
dropout 率, 不使用掩码
66         self.__setattr__('dec_vanilla_attention_%d' % i,
-----)
67         #TODO: 调用前馈神经网络模块进行构建, 包含一个隐藏层, 中间层维度通常为输入hidden_units的4
倍, 输出层维度恢复到输入
68         self.__setattr__('dec_feed_forward_%d' % i,
-----)
69     self.logits_layer = nn.Linear(self.hp.hidden_units, self.dec_voc)
70     #TODO: 调用标签平滑模块
71     self.label_smoothing = _____
72     # self.losslayer = nn.CrossEntropyLoss(reduce=False)
73     print("LabelSmoothing PASS!")
74 def forward(self, x, y):
75     # define decoder inputs
76     # self.decoder_inputs = torch.cat([Variable(torch.ones(y[:, :1].size()).cuda() * 2).long(), y
[:, :-1]], dim=-1) # 2:<S>
77     input_tensor = torch.ones(y[:, :1].size())

```

```

78     if (x.is_cuda):
79         input_tensor = input_tensor.cuda()
80     if (x.device.type == 'mlu'):
81         #TODO: 将input_tensor放在寒武纪卡上
82         input_tensor = -----
83     self.decoder_inputs = torch.cat([Variable(input_tensor * 2).long(), y[:, :-1]], dim=-1) # 2:<
S>

84
85     # Encoder
86     #TODO: 通过编码器端的词嵌入
87     self.enc = -----
88     # Positional Encoding
89     #TODO: 使用正弦位置编码
90     if self.hp.sinusoid:
91         self.enc += -----
92     else:
93         #self.enc += self.enc_positional_encoding(
94         #    Variable(torch.unsqueeze(torch.arange(0, x.size()[1]), 0).repeat(x.size(0), 1).long()
95         #.cuda()))
96         enc_positional = torch.unsqueeze(torch.arange(0, x.size()[1]), 0).repeat(x.size(0), 1).
long()
97         if (x.is_cuda):
98             enc_positional = enc_positional.cuda()
99         if (x.device.type == 'mlu'):
100             enc_positional = enc_positional.to('mlu')
101         self.enc += self.enc_positional_encoding(Variable(enc_positional))
102
103     #TODO: Dropout 正则化
104     self.enc = -----
105     # Blocks
106     for i in range(self.hp.num_blocks):
107         #实现编码器的 self_attention 机制
108         self.enc = self.__getattr__('enc_self_attention_%d' % i)(
109             -----)
110         #实现编码器的Feed Forward机制
111         self.enc = self.__getattr__('enc_feed_forward_%d' % i)(
112             -----)
113     # Decoder
114     #TODO: 解码器端的词嵌入
115     self.dec = -----
116     # Positional Encoding
117     #TODO: 使用正弦位置编码
118     if self.hp.sinusoid:
119         self.dec += -----
120     else:
121         #self.dec += self.dec_positional_encoding(
122         #    Variable(torch.unsqueeze(torch.arange(0, self.decoder_inputs.size()[1]), 0).repeat(
123         #self.decoder_inputs.size(0), 1).long().cuda()))
124         dec_positional = torch.unsqueeze(torch.arange(0, self.decoder_inputs.size()[1]), 0).repeat
(self.decoder_inputs.size(0), 1).long()
125         if (x.is_cuda):
126             dec_positional = dec_positional.cuda()
127         if (x.device.type == 'mlu'):
128             dec_positional = dec_positional.to('mlu')
129         #TODO: 加上位置编码
130         self.dec += -----
131
132     #TODO: 进行Dropout
133     self.dec = -----

```

```

133 # Blocks
134 for i in range(self.hp.num_blocks):
135     # 实现解码器内部self-attention机制
136     self.dec = self.__getattr__('dec_self_attention_%d' % i)(self)
137     # 实现解码器内部vanilla attention机制
138     self.dec = self.__getattr__('dec_vanilla_attention_%d' % i)(self)
139     # 实现解码器的feed forward
140     self.dec = self.__getattr__('dec_feed_forward_%d' % i)(self)
141
142 # Final linear projection
143 #TODO: 线性映射到词表维度
144 self.logits = self.dec(self)
145 #TODO: 计算概率分布, 并展平成二维(batch_size * seq_len, vocab_size)
146 self.probs = self.logits.softmax(-1)
147 #TODO: 获取最大概率对应的预测词索引
148 _, self.preds = self.probs.max(-1)
149 self.istarget = (1. - y.eq(0.).float()).view(-1)
150 self.acc = torch.sum(self.preds.eq(y).float()).view(-1) * self.istarget / torch.sum(self.istarget)
151
152 # Loss
153 # self.y_onehot = torch.zeros(self.logits.size()[0] * self.logits.size()[1], self.dec_voc).
154 cuda()
155 self.y_onehot = torch.zeros(self.logits.size()[0] * self.logits.size()[1], self.dec_voc)
156 if (x.is_cuda):
157     self.y_onehot = self.y_onehot.cuda()
158 if (x.device.type == 'mlu'):
159     self.y_onehot = self.y_onehot.to('mlu')
160
161 self.y_onehot = Variable(self.y_onehot.scatter_(1, y.view(-1, 1).data, 1))
162 #TODO: 调用标签平滑模块, 得到平滑后的标签分布
163 self.y_smoothed = self.y_onehot * self.hp.label_smoothing + self.logits.softmax(-1) * (1 - self.hp.label_smoothing)
164
165 # self.loss = self.losslayer(self.probs, self.y_smoothed)
166 self.loss = - torch.sum(self.y_smoothed * torch.log(self.probs), dim=-1)
167 # print(self.loss)
168
169 self.mean_loss = torch.sum(self.loss * self.istarget) / torch.sum(self.istarget)
170
171 return self.mean_loss, self.preds, self.acc

```

4.5.5.3 Transformer 翻译模型评估

Transformer 翻译模型评估主要由环境与依赖导入模块、模型推理与评估模块、程序入口与参数配置模块组成。

1) 环境与依赖导入模块

环境与依赖导入模块的代码示例如4.25所示, 用于导入所需的标准库、PyTorch 相关模块、数据加载与模型结构定义文件。

代码示例 4.25 环境与依赖导入模块

```

1 ## file: eval.py
2 # -*- coding: utf-8 -*-
3 from __future__ import print_function
4 import argparse
5 import codecs
6 import os

```

```

7 import random
8 import time
9 import numpy as np
10
11 import torch
12 from hyperparams import Hyperparams as hp
13 from data_load import TestDataSet, load_de_vocab, load_en_vocab
14 from nltk.translate.bleu_score import corpus_bleu
15 from AttModel import AttModel
16 from torch.autograd import Variable

```

2) 模型推理与结果评估模块

模型推理与结果评估模块如代码示例4.26所示。该模块加载测试集和模型权重，执行自回归式的翻译生成，并将结果输出到日志文件。随后根据参考答案计算 BLEU 分数，用于衡量模型翻译质量，同时记录推理时间和吞吐率，辅助分析模型性能。

代码示例 4.26 模型推理与结果评估模块

```

1  ## file: eval.py
2  def eval(args):
3      # Load data
4      if args.seed is not None:
5          torch.manual_seed(args.seed)
6          np.random.seed(args.seed)
7          random.seed(args.seed)
8
9      #TODO: 调用函数加载德语词表
10     de2idx, idx2de = _____
11     #TODO: 调用函数加载英语词表
12     en2idx, idx2en = _____
13     enc_voc = len(de2idx)
14     dec_voc = len(en2idx)
15
16     #TODO: 初始化Transformer翻译模型
17     model = _____
18     print("AttModel PASS!")
19
20     source_test = args.dataset_path + hp.source_test
21     target_test = args.dataset_path + hp.target_test
22     #TODO: 构建测试数据集对象
23     test_dataset = _____
24     #TODO: 使用PyTorch的 DataLoader对测试集进行批量加载
25     test_loader = _____(
26         test_dataset,
27         batch_size=_____,
28         shuffle=False,
29         num_workers=_____,
30         pin_memory=False)
31
32     if args.device == "MLU":
33         #model.to(ct.mlu_device())
34         model.mlu()
35     elif args.device == "GPU":
36         model.cuda()
37     #TODO: 从指定路径加载预训练模型参数
38     state = _____(_____, map_location='cpu')
39     #TODO: 将加载的模型参数赋值到当前模型中，以完成模型权重的恢复
40     _____
41
42     print('Model Loaded.')
43     #TODO: 设置模型为评估模式

```

```

44 -----
45
46 #TODO: 以UTF-8编码、追加模式打开指定日志文件 args.log_path, 用于记录模型评估的输出结果
47 with open(args.log_path, 'a', 'utf-8') as fout:
48     list_of_refs, hypotheses = [], []
49     t1 = time.time()
50     #TODO: 遍历测试集, 每次获取一个批次的输入数据、原始句子和目标句子
51     for i, (source, target, pred) in enumerate(test_loader):
52         if (i == args.iterations):
53             break
54         # Autoregressive inference
55         if args.device == "GPU":
56             x_ = x.long().cuda()
57             preds_t = torch.LongTensor(np.zeros((x.size()[0], hp.maxlen), np.int32)).cuda()
58             preds = Variable(preds_t).cuda()
59         elif args.device == "MLU":
60             x_ = x.long().to('mlu')
61             preds_t = torch.LongTensor(np.zeros((x.size()[0], hp.maxlen), np.int32)).to('mlu')
62             preds = Variable(preds_t.to('mlu'))
63         else:
64             x_ = x.long()
65             preds_t = torch.LongTensor(np.zeros((x.size()[0], hp.maxlen), np.int32))
66             preds = Variable(preds_t)
67
68         for j in range(hp.maxlen):
69             _preds, _ = model(x_, preds)
70             preds_t[:, j] = _preds.data[:, j]
71             preds = Variable(preds_t.long())
72         preds = preds.data.cpu().numpy()
73
74         # Write to file
75         for source, target, pred in zip(sources, targets, preds): # sentence-wise
76             got = " ".join(idx2en[idx] for idx in pred).split("</S>")[0].strip()
77             fout.write("- source: " + source + "\n")
78             fout.write("- expected: " + target + "\n")
79             fout.write("- got: " + got + "\n\n")
80             fout.flush()
81
82         # bleu score
83         ref = target.split()
84         hypothesis = got.split()
85         if len(ref) > 3 and len(hypothesis) > 3:
86             list_of_refs.append([ref])
87             hypotheses.append(hypothesis)
88     #TODO: 计算整个推理过程所消耗的总时间
89     temp_time = time.time()
90     print("time:", temp_time)
91     #TODO: 计算每秒处理的样本数 (吞吐量)
92     print("qps:", len(list_of_refs) / (temp_time - t1))
93     #TODO: 计算模型翻译结果与参考答案之间的BLEU评分, 用于评价翻译质量
94     score = bleu_scorer(list_of_refs, hypotheses)
95     fout.write("Bleu Score = " + str(100 * score))
96     print("Bleu Score = {}".format(100 * score))
97 if os.getenv('AVG_LOG'):
98     with open(os.getenv('AVG_LOG'), 'a') as train_avg:
99         train_avg.write('Bleu Score: {}\n'.format(100 * score))
100 print("Eval PASS!")

```

3) 程序入口与参数配置模块

程序入口与参数配置模块的代码示例如4.27所示。它负责启动程序, 解析命令行参数, 设置设备相关依赖, 然后调用 eval 函数执行模型评估流程。

代码示例 4.27 程序入口与参数配置

```
1 ## file: eval.py
2 if __name__ == '__main__':
3     #TODO: 创建命令行参数解析器
4     parser = argparse.ArgumentParser(description="Transformer evaluation.")
5     parser.add_argument('--device', default='MLU', type=str, help='set the type of hardware used for
6     evaluation.')
7     parser.add_argument('--seed', default=0, type=int, help='random seed')
8     parser.add_argument('--pretrained', default='model_epoch_20.pth', type=str, help='training ckps
9     path')
10    parser.add_argument('--batch-size', default=32, type=int, help='evaluation batch size.')
11    parser.add_argument('--workers', default=4, type=int, help='number of workers.')
12    parser.add_argument('--log-path', default='output.txt', type=str, help='evaluation file path.')
13    parser.add_argument('--dataset-path', default='corpora/', type=str, help='The path of imagenet
14    dataset.')
15    parser.add_argument('--iterations', default=-1, type=int, help="Number of training iterations.")
16    parser.add_argument('--bitwidth', default=8, type=int, help="Set the initial quantization width of
17    network training.")
18    parser.add_argument('--opt_level', type=str, default="O0", help='choose level of mixing precision'
19    )
20    #TODO: 解析命令行输入的参数
21    args = parser.parse_args()
22
23    if args.device == "MLU":
24        import torch_mlu
25
26    #TODO: 调用 eval 函数开始模型评估流程
27
28    if args.device == "MLU":
29        print("Transformer MLU PASS!")
30    else:
31        print("Transformer CPU PASS!")
```

4.5.5.4 实验运行

1) 环境申请

申请实验环境并登录云平台,本实验的代码存放在云平台/opt/code_chap_4/exp_4_5_transformer/目录下。

```
# 登录云平台
ssh root@xxx.xxx.xxx.xxx -p xxxxxx
# 进入 /opt/code_chap_4/exp_4_5_transformer/ 目录
cd /opt/code_chap_4/exp_4_5_transformer/
```

2) 代码实现

补全 modules.py、AttModel.py、eval.py 文件。

```
# 补全 modules.py 文件
vim modules.py
# 补全 AttModel.py 文件
vim AttModel.py
# 补全 eval.py 文件
vim eval.py
```

3) 运行实验

分别在 CPU 和 DLP 上运行 Transformer。

```
# 在CPU上运行Transformer
bash run_transformer_cpu.sh
# 在DLP上运行Transformer
bash run_transformer_mlu.sh
```

4.5.6 实验评估

实验评分标准为：

- 60 分标准：正确实现 Transformer 词嵌入模块、层归一化模块、位置编码模块、多头注意力机制模块、前馈神经网络模块、标签平滑模块一系列基本单元模块；
- 80 分标准：在 60 分的基础上，正确实现 Transformer 翻译模型构建；
- 90 分标准：在 80 分的基础上，正确实现 Transformer 翻译模型评估模块；
- 100 分标准：在 90 分的基础上，在 CPU 和 DLP 上正确实现 Transformer。

4.5.7 实验思考

- 1) 为什么 Transformer 完全摒弃了 CNN 和 RNN 结构？相比它们有什么本质优势？
- 2) 多头注意力机制为何被设计为并行的多个注意力头，而不是通过扩展维度实现一个更大的单一注意力？不同注意力头在表达能力上分别起到什么作用？
- 3) Transformer 在计算注意力时，为什么要在 Softmax 之前除以 $\sqrt{d}$ ？
- 4) Transformer 最初用于自然语言处理，后来被成功应用到计算机视觉（Vision Transformer）、音频、图数据等领域。Transformer 为何具有如此强的跨模态迁移能力？
- 5) Transformer 的运行有哪些核心瓶颈？在计算、存储和推理效率上分别存在什么挑战？